CIS 3300 · THE SPEEDRUN LEADERBOARD

Checkpoint 1

The File Problem and the Run Record

Starts from: SpeedrunStarter. **Ends at:** SpeedrunCheckpoint1. **Prepares you for:** Lab W14, Tasks 1 and 2.

This checkpoint fixes the missing-file problem from Step 3 of the Starter Program, in *Getting Started*, and gives each run a record of its own that refuses bad values. It does not yet fix the crash from the Starter Program's Step 2. That is Checkpoint 2.

Before you start, see the *Getting Started* guide for how to import a project into Eclipse and for the rules the leaderboard uses for each field.

A note about this guide. I developed this guide with help from AI, which drafted much of the code, data, and explanations, and I review and revise them. Every program was compiled and run, and every output shown is real. If something looks wrong, does not match what you see in Eclipse, or contradicts the book or the notes, tell me. This guide shows how to build one program. It does not replace the notes, the assigned chapters, or the recorded lectures.

Step 1: Why `throws` is not enough

The starter's `main` method is declared with `throws IOException`. A `throws` clause does not handle an exception. It passes the exception to whoever called the method. Nobody in your program calls `main`: Java does. So when an `IOException` leaves `main`, Java handles it the only way it can, by printing the stack trace and stopping. `main` is the top of the program, the last place your code can deal with the problem, so it must *catch* the exception instead.

Step 2: Catch the exception

Remove `throws IOException` from `main`, put the file name in a variable, and wrap the body of `main` in a `try` block with two `catch` blocks. Add `import java.nio.file.NoSuchFileException;` at the top of the file.

```
public static void main(String[] args) {
    String fileName = "files/runs_clean.csv";
    try {
        List<String> lines = Files.readAllLines(Path.of(fileName));

        // ... the for loop, exactly as before ...

    } catch (NoSuchFileException e) {
        System.out.println("Could not find the file " + e.getFile());
    } catch (IOException e) {
        System.out.println("Could not read the file " + fileName + ": "
                + e.getMessage());
    }
}
```

- **Why two `catch` blocks?** A missing file is the most likely problem, and it deserves the clearest message. `NoSuchFileException` is a subclass of `IOException`, and its `getFile()` method returns the file's name. The second block catches every other `IOException`, such as a file you do not have permission to read.
- **Why this order?** Java tries the `catch` blocks from the top. If `IOException` came first, it would catch every `NoSuchFileException` too, and the second block could never run. The compiler refuses that order:

  ```
  error: exception NoSuchFileException has already been caught
  ```

  Always put the more specific exception first.
- **Why the `fileName` variable?** The second message needs the name, and an `IOException` does not always include it.

Step 3: Test it

Change the file name to `run_clean.csv` again, and run it:

```
Could not find the file files/run_clean.csv
```

One clear line, and no stack trace. Change the name back to `runs_clean.csv` and run it again. The output is the same as in Step 1 of the Starter Program.

Step 4: Write the Run record

Right now a run is six separate variables inside a loop. It should be one object. In the `speedrun` package, create a new *record* named `Run`. In Eclipse, choose **File**, **New**, **Record**. A record is declared with the keyword `record`, not `class`, and it is *immutable*: once a `Run` is created, its values can never change. So if the values are checked once, when the record is created, every `Run` that exists is good.

```
package speedrun;

import java.time.Duration;
import java.time.LocalDate;
import java.util.Objects;

public record Run(String player, String game, String category, Duration time,
        LocalDate date, boolean verified) {

    public Run {
        requireText(player, "player");
        requireText(game, "game");
        requireText(category, "category");
        Objects.requireNonNull(time, "time is required");
        Objects.requireNonNull(date, "date is required");
        if (time.isNegative() || time.isZero()) {
            throw new IllegalArgumentException(
                    "time must be greater than 0, but was " + RunTime.format(time));
```

```
        }
        if (date.isAfter(LocalDate.now())) {
            throw new IllegalArgumentException("date " + date + " is in the future");
        }
    }

    private static void requireText(String value, String name) {
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException(name + " is required");
        }
    }
}
```

Here is what each part does.

- **The header** lists the record's *components*. From that one line, Java writes the private final fields, a constructor that takes all six values, an accessor method for each component, and `equals`, `hashCode`, and `toString`. The accessors are named after the components, so you write `run.time()`, not `run.getTime()`.
- **The compact constructor**, `public Run { ... }`, has no parameter list. It runs before the values are stored, and Java stores them for you afterward, so you never write `this.time = time`. Its only job is to check the values.
- **A `throw` statement** stops the constructor. `new IllegalArgumentException(...)` creates the exception object, with a message that names the problem and includes the bad value. `throw` sends it to the caller immediately. The rest of the constructor does not run, and the `Run` is never created.
- **No `throws` clause is needed,** because `IllegalArgumentException` is unchecked. It means "the caller passed a bad value," and the compiler does not force anyone to plan for it.
- **`Objects.requireNonNull`** throws a `NullPointerException` with your message if the value is `null`, in one line.
- **`requireText`** is a private helper, so the three text checks are written once. A record may have its own methods, as long as they are not extra fields.
- **`LocalDate.now()`** is today's date, so a run dated after today is rejected.

Step 5: Try the record

Before using `Run` in the loop, try it on its own. Add these lines at the very top of `main`, and add `import speedrun.Run;` with the other imports:

```
Run good = new Run("Kestrel", "Celeste", "Any%", RunTime.parse("26:41.6"),
        LocalDate.of(2026, 2, 14), true);
System.out.println(good);
Run bad = new Run("Moss", "Portal", "Inbounds", RunTime.parse("0:00.0"),
        LocalDate.of(2025, 11, 14), true);
System.out.println(bad);
```

```
Run[player=Kestrel, game=Celeste, category=Any%, time=PT26M41.6S, date=2026-02-14, verified=true
    ]
Exception in thread "main" java.lang.IllegalArgumentException: time must be greater than 0, but
    was 0:00.0
        at speedrun.Run.<init>(Run.java:32)
        at app.SpeedrunReport.main(SpeedrunReport.java:27)
```

The good run prints with the `toString` that Java wrote. Notice how a `Duration` prints itself: `PT26M41.6S`, meaning a period of time of 26 minutes and 41.6 seconds. That is why the program uses `RunTime.format` instead. The bad run never exists: the constructor threw, and `<init>` in the trace is how Java names a constructor. Delete the six lines before going on.

Step 6: Use the record in the loop

Replace the six variables in the loop with one `Run`, and print it with its accessors:

```
for (int i = 1; i < lines.size(); i++) {
    String[] fields = lines.get(i).split(",");
    Run run = new Run(fields[0], fields[1], fields[2],
            RunTime.parse(fields[3]), LocalDate.parse(fields[4]),
            fields[5].equals("yes"));

    System.out.printf("%-10s %-36s %-8s %10s  %s  %s%n", run.player(),
            run.game(), run.category(), RunTime.format(run.time()),
            run.date(), run.verified() ? "verified" : "unverified");
}
```

Run it. The output is the same as in Step 1 of the Starter Program. Now change the file name to `runs.csv` and run it again. It still crashes on line 7, with the same `NumberFormatException` as before. Checkpoint 1 made bad *runs* impossible, but not bad *lines*: `RunTime.parse` fails before the `Run` constructor even runs, and if it had not, the constructor would throw instead. Either way, one exception still stops the whole loop. Change the file name back to `runs_clean.csv`.

Your project should now match SpeedrunCheckpoint1.

Check yourself

1. What does Java do with an exception that leaves `main`, and why is that a problem?
2. Why must the `catch` for `NoSuchFileException` come before the `catch` for `IOException`?
3. After this checkpoint, can a `Run` with a time of `0:00.0` exist anywhere in the program? Why or why not?
4. Line 31 of `runs.csv` has `maybe` in the verified field. What does the program do with it right now?

Answers

1. Java prints the stack trace and stops the program. The person running the program sees a wall of text meant for programmers, and nothing after the error runs.

2. `NoSuchFileException` is a subclass of `IOException`, so a `catch` for `IOException` placed first would catch it too, and the more specific block could never run. The compiler rejects that order.

3. No. The compact constructor throws before the record is created, and a record's values can never change afterward, so every `Run` that exists has a time greater than 0.

4. It quietly treats the run as unverified, because `fields[5].equals("yes")` is `false` for anything except `yes`. That is worse than a crash: the bad value is hidden. Checkpoint 2 makes it a bad line.